manvendra@rajpoot.dev
+91 94522 77560

Manvendra Rajpoot

rajpoot.dev
github.com/alzywelzy
linkedin.com/in/alzywelzy

Dear Hiring Manager,

I am writing to apply for the Python Developer role on your team. Python is the language I have shipped the most production code in, across Django, Django REST Framework and FastAPI, async SQLAlchemy, Celery and Pydantic, and I care about writing it typed, tested and easy for the next person to change.

At CloudTechTiq I architected a multi-tenant hosting and billing/ERP platform on Django REST Framework spanning more than 25 apps and 230 ViewSets. I designed its dual-gateway payment system across Razorpay and Stripe with HMAC-SHA256 webhook idempotency, wired through invoicing, provisioning and double-entry ledger reconciliation, and Indian GST e-invoicing against the NIC gateway with IRN generation and signed QR codes. I then extracted billing into a FastAPI microservice that is the single billing authority for the platform, with a double-entry ledger, multi-invoice payment allocation and refunds, and multi-entity issuance that resolves entity, currency and tax from the billing address.

I built an OAuth2/OpenID authorization server and FastAPI gateway to Apache CloudStack on async SQLAlchemy 2, asyncpg and Alembic, with webhook-first state sync, a polling fallback and job progress streamed over Server-Sent Events. Across the platform I implemented RS512 JWT with JWKS and refresh rotation, TOTP two-factor sign-in, a 43-permission RBAC, Django Channels WebSockets, and a 24-task Celery system with late acknowledgement and worker-lost requeue. I eliminated N+1 queries with Prefetch, OuterRef and Exists, added keyset pagination, and instrumented everything with OpenTelemetry. I also open-sourced a type-safe Namecheap SDK on httpx and Pydantic v2, mypy-strict and published to PyPI.

I also maintain open-source Python services: Shortly, a FastAPI URL shortener with a Redis read-through cache and Docker Compose and systemd deployment, and fastapi-genesis, a feature-sliced FastAPI starter on async SQLAlchemy 2, Alembic, uv, ruff and ty. I plan migrations with written runbooks that carry a check and a rollback for every step, gate releases on a test-parity check so every existing pytest is ported or deliberately dropped, and design scheduled and queued work with leases, retries and dead-letter queues so a failed job is visible rather than lost.

I like Python codebases where routers stay thin, logic lives in services, settings come from one place and tests run without a database, and I bring that discipline to every team I join. I review for clarity as much as correctness, and I prefer small, typed, well-tested changes that are easy to reason about months later.

I would welcome the chance to discuss how this experience fits your team, and I am happy to walk through any of the systems above in technical detail. Thank you for your time and consideration.

Sincerely,

Manvendra Rajpoot